MedAudit

Submitted in partial fulfillment of the requirements of the degree

BACHELOR OF ENGINEERING IN

ARTIFICIAL INTELLIGENCE & DATA SCIENCE ENGINEERING

By

Ketaki Mahadik (31)

Vedant Mahamunkar (32)

Jay Nagavkar (39)

Guide Name

Prof. Bhagyashri Kulkarni



Department of Artificial Intelligence & Data Science Engineering

A.C.Patil College of Engineering

Kharghar, Navi Mumbai

University of Mumbai

(AY 2026-27)

CERTIFICATE

This is to certify that the Mini Project entitled **" MedAudit — Permissioned Healthcare Ledger & Access Control Portal "** is a bonafide work of **Ketaki Mahadik (31) , Vedant Mahamunkar (32) & Jay Nagavkar (39)** submitted to the University of Mumbai in partial fulfillment of the requirement for the award of the degree of **"Bachelor of Engineering"** in **"Artificial Intelligence & Data Science Engineering" .**

Prof.Bhagyashri Kulkarni
Guide

Prof. S. P.Bansu
HOD

Dr. S. R. Deore
Principal

Mini Project Approval

This Mini Project entitled "**MedAudit — Permissioned Healthcare Ledger & Access Control Portal"** by **Ketaki Mahadik (31) , Vedant Mahamunkar (32) & Jay Nagavkar (39)** is approved for the degree of **Bachelor of Engineering** in **Artificial Intelligence & Data Science Engineering.**

Examiners

1……
(Internal Examiner Name & Sign)

2……
(External Examiner name & Sign)

Date:

Place:

1. Introduction

The rapid digitization of healthcare systems globally has led to an exponential increase in the generation and exchange of Electronic Health Records (EHR). While this digital transformation has improved care coordination, it has simultaneously introduced severe privacy and security vulnerabilities. Traditional centralized database architectures act as single points of failure, making them highly susceptible to ransomware attacks, unauthorized third-party data mining, and insider threats. Furthermore, conventional database audit logs (such as MySQL or MongoDB access tables) are inherently mutable; privileged system administrators can retroactively alter or wipe access histories, resulting in an opacity of access logs and a failure to meet strict non-repudiation standards.

While blockchain technology offers a solution to data immutability, public blockchains like Ethereum or Bitcoin are fundamentally unsuited for healthcare applications. They suffer from high transaction latency, volatile gas fees, and critical compliance violations—most notably, the inability to guarantee privacy, as storing any Protected Health Information (PHI) on a public ledger directly violates regulations such as HIPAA and the GDPR.

The proposed MedAudit system resolves this critical industry paradox by introducing an enterprise-grade, permissioned healthcare access control and audit trail platform. Utilizing a consortium model based on Hyperledger Fabric 2.5, MedAudit creates a secure, decentralized trust network between participating hospitals, clinics, and patients without relying on public consensus mechanisms or cryptocurrency tokens.

To ensure absolute privacy and regulatory compliance, the MedAudit architecture strictly separates data into two distinct operational boundaries. The off-chain storage layer acts as a secure repository where diagnostic medical files are encrypted server-side utilizing AES-256-GCM envelope encryption with unique, per-record Data Encryption Keys (DEKs). Conversely, the on-chain ledger governs the access control logic and stores only immutable cryptographic SHA-256 digests of the files alongside transaction audit events. This dual-layer approach guarantees that zero PHI is ever exposed to the blockchain network.

Ultimately, MedAudit shifts the paradigm of medical data management from a hospital-centric model to a patient-centric one. Patients maintain absolute, cryptographic ownership over their health records. Through the portal, patients can grant time-bound, purpose-specific access to designated medical professionals and retain the power to instantly revoke these permissions. Every phase of the data lifecycle—including access requests, approvals, denials, revocations, and successful file retrievals—is irrevocably logged with a unique transaction ID, establishing a transparent, unforgeable history of medical data interactions.

2. Objectives

The primary objective of the MedAudit project is to engineer a highly secure, decentralized application (DApp) that establishes an immutable, non-repudiable audit trail for diagnostic medical records while strictly guaranteeing patient data privacy and zero public data leakage.

To achieve this overarching goal, the project targets the following specific objectives:

1. **Establish Patient-Centric Access Control:** Develop a decentralized consent management system that empowers patients to explicitly review, approve, deny, or revoke medical record access requests from healthcare professionals in real time.
2. **Implement a Zero-Knowledge On-Chain Boundary:** Architect a dual-layer data storage mechanism that strictly guarantees that no raw medical files, patient identities, or Protected Health Information (PHI) are ever stored on the blockchain ledger, ensuring full compliance with HIPAA and GDPR regulations.
3. **Enforce End-to-End Cryptographic Integrity:** Utilize AES-256-GCM authenticated encryption for securing all medical payloads in the off-chain relational database, while dynamically computing and anchoring SHA-256 cryptographic hashes onto the Fabric ledger during file registration.
4. **Automate Cryptographic Verification on Retrieval:** Build a secure download mechanism that re-computes the SHA-256 hash of decrypted files upon doctor retrieval and matches it against the immutable on-chain anchor, immediately aborting the transfer if a single byte has been tampered with.
5. **Guarantee Non-Repudiation and Comprehensive Auditability:** Program smart contracts (chaincode) to irrevocably log all access lifecycle events (requests, approvals, downloads, revocations) with precise timestamps, actor IDs, and transaction IDs (txId) to produce a tamper-resistant historical audit trail.
6. **Implement Secure Role-Based Operations:** Design a robust JWT-authenticated backend routing system that explicitly restricts application interactions based on registered roles, ensuring only verified doctors can request access and only legitimate patient owners can approve those requests.
7. **Eliminate Transaction Costs and Processing Latency:** Utilize Hyperledger Fabric's permissioned Raft consensus mechanism to execute access control logic deterministically and instantly, bypassing the high latency and fluctuating gas fees associated with public EVM-compatible blockchains.

3. Problem Statement

Current cloud-based centralized hospital databases present several structural and cryptographic flaws that jeopardize patient privacy. In a traditional architecture, Electronic Health Records (EHR) are stored in centralized relational databases (such as MySQL or Oracle). These architectures inherently act as single points of failure. They are prime targets for ransomware attacks; if a malicious actor breaches the central server, the entire hospital's database can be encrypted or exfiltrated. Furthermore, traditional database audit logs (e.g., MongoDB access tables) are entirely mutable. Privileged system administrators or attackers with escalated privileges can retroactively alter, forge, or wipe access histories to cover their tracks, resulting in an opacity of access logs.

Additionally, patients rarely have real-time visibility or cryptographic control over which specialists, external clinics, or insurance providers are accessing their highly sensitive records. When hospitals attempt to solve data immutability using public blockchains (like Ethereum), they introduce critical compliance violations. Public blockchains replicate all ledger data across thousands of global nodes. Storing Protected Health Information (PHI) on these networks directly violates the General Data Protection Regulation (GDPR)—specifically the "Right to be Forgotten"—and the Health Insurance Portability and Accountability Act (HIPAA), which mandate strict data localization and privacy. Furthermore, the variable "gas fees" and high latency associated with public consensus mechanisms make them economically and operationally unviable for high-throughput clinical environments.

To meet these conflicting regulatory standards, access control must be strictly authenticated, purpose-restricted, time-limited, and audited via an unforgeable ledger, while keeping the raw data entirely private. MedAudit addresses this industry paradox by utilizing a permissioned consortium model using Hyperledger Fabric. This eliminates single points of failure, completely bypasses public gas fees, guarantees non-repudiation of access logs, and ensures that zero PHI is ever exposed to the blockchain network.

4. Tools and Technologies Used

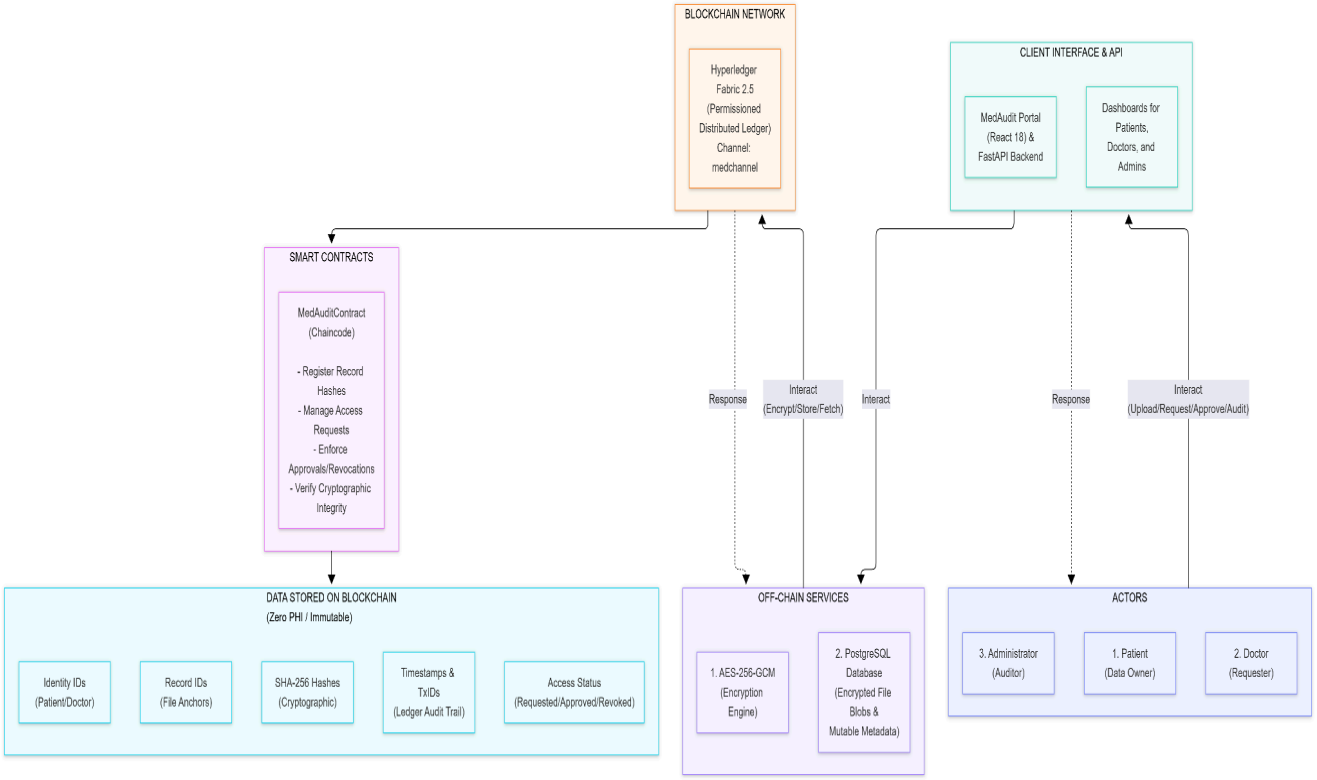
Tool/Technology	Purpose / Description
Hyperledger Fabric 2.5	A permissioned distributed ledger technology (DLT) designed specifically for enterprise consortiums. It supports private data collections, X.509 certificate-based identity management, and channel-based architecture. It uses a deterministic Raft consensus protocol rather than

	computationally expensive Proof-of-Work, allowing for high transaction throughput with zero transaction costs (gas fees).
Python (FastAPI)	A modern, high-performance web framework for building APIs with Python 3.11. Chosen for its native asynchronous support (ASGI) and automatic OpenAPI documentation generation. It handles computationally intensive tasks: executing the AES-256-GCM encryption/decryption pipelines, managing OAuth2/JWT user authentication, and routing requests securely between the client frontend and the blockchain gateway.
Node.js & @hyperledger/fabric-gateway	A dedicated microservice acting as the critical bridge between the Python backend and the Hyperledger Fabric network. Utilizing the official Fabric Gateway SDK, it translates RESTful HTTP requests into gRPC transactions, handling transaction proposal, endorsement collection from peer nodes, and submission to the ordering service.
PostgreSQL	A robust open-source relational database utilized as the off-chain storage layer to maintain strict compliance with privacy laws (ensuring zero medical data is stored on the blockchain). It stores mutable user metadata (protected by Bcrypt hashing algorithms) and securely houses the heavy AES-256-GCM encrypted file blobs.

React 18 & Tailwind CSS	Used to build the client interface as a highly responsive Single Page Application (SPA) using TypeScript. It provides distinct, secure dashboards dynamically rendered based on user roles (Patient, Doctor, Administrator). Tailwind CSS builds a utility-first user interface that clearly displays cryptographic verification badges, pending access requests, and complex audit timelines.
AES-256-GCM & SHA-256	Advanced Encryption Standard in Galois/Counter Mode (AES-256-GCM) is the authenticated envelope encryption algorithm used to secure medical files off-chain, ensuring both confidentiality and data authenticity. SHA-256 (Secure Hash Algorithm 256-bit) is the cryptographic hashing algorithm used to anchor file integrity onto the immutable blockchain state.

5. System Architecture

The MedAudit architecture implements a strict, zero-trust boundary separation between highly sensitive medical files (stored off-chain) and access control metadata (stored on-chain). This hybrid design ensures maximum privacy while maintaining decentralized trust.



5.1 Architectural Breakdown

- **Presentation Layer:** This layer is responsible for client-side interactions and session management. It features distinct role-tailored views. Patients have interfaces for uploading files and approving/revoking access. Doctors are provided with interfaces for requesting access and retrieving authorized files. Administrators have access to the global ledger audit explorer.
- **Application Layer (Backend):** The FastAPI server acts as the primary orchestrator. It intercepts incoming HTTP requests, validates JSON Web Tokens (JWT) for authentication, enforces Role-Based Access Control (RBAC), and manages the memory-intensive AES-256-GCM encryption and decryption algorithms before passing metadata to the blockchain layer.
- **Gateway Microservice:** Built on Node.js, this layer abstracts the complexities of blockchain transaction lifecycles. It connects to the Fabric network over secure gRPC, evaluates smart contracts, and returns the deterministic execution results back to the Python application layer.
- **On-Chain Distributed Ledger:** Smart contracts (Chaincode) are deployed on a private communication channel named `medchannel`. This channel maintains the global state across participating organizations (e.g., `HospitalOrg` and `PatientOrg`) using a deterministic Raft consensus mechanism, ensuring all nodes in the network agree on the sequence of access

events.

5.2 Data Boundary Separation

- **On-Chain Ledger (`medchannel`):** The blockchain acts purely as an access control and verification engine. It stores discrete metadata: Record IDs, File Hashes (SHA-256), Patient IDs, Requester IDs, Expiration Timestamps, Action Types (e.g., APPROVED, REVOKED), and Transaction IDs (TxIDs). The ledger is strictly append-only and cryptographically immutable. Absolutely zero Protected Health Information (PHI) is stored here.
- **Off-Chain Storage (Encrypted DB):** The PostgreSQL database acts as the secure vault for the actual medical data. It stores User Credentials, Record Titles/Descriptions, and the AES-256-GCM Encrypted File Blobs. While this data is technically mutable, the ciphertext is entirely unreadable without the corresponding cryptographic master keys, neutralizing the threat of database breaches.

5.1 Architectural Breakdown

- **Presentation Layer:** Features distinct role-tailored views for Patients (uploading, approving/revoking), Doctors (requesting, reading), and Admins (auditing).
- **Application Layer:** A FastAPI backend manages OAuth2/JWT authentication, AES-256-GCM encryption, and Role-Based Access Control.
- **Gateway Microservice:** A Node.js bridge using the Fabric Gateway SDK connects the REST APIs to the blockchain over gRPC.
- **On-Chain Distributed Ledger:** Smart contracts deployed on the `medchannel` channel maintain the state across `HospitalOrg` and `PatientOrg` using a deterministic Raft consensus mechanism.

5.2 Data Boundary Separation

- **On-Chain Ledger (`medchannel`):** Stores Record IDs, File Hashes (SHA-256), Patient IDs, Requester IDs, Expirations, Actions, and TxIDs. The ledger is append-only and cryptographically immutable. Zero PHI is stored here.
- **Off-Chain Storage (Encrypted DB):** Stores User Credentials (Bcrypt), Record Titles/Descriptions, and AES-256-GCM Encrypted File Blobs. The data is mutable but the ciphertext is unreadable without the master key.

6. Implementation Steps

Step 1: User Registration & Identity Initialization Users register via the `/api/v1/auth/register` endpoint. The backend validates the input payload, assigns strict hierarchical roles (patient, doctor, admin), and secures passwords using the bcrypt hashing algorithm with a high work factor (salt rounds). Upon successful login, the system issues a cryptographically signed JSON Web Token (JWT) that must be passed in the authorization header of all subsequent requests.

Step 2: Record Encryption & Cryptographic Anchoring When a patient uploads a raw medical diagnostic file via the `/api/v1/records` endpoint, the data never touches the disk in plaintext. The backend generates a cryptographically secure random 256-bit Data Encryption Key (DEK) and a 96-bit Initialization Vector (IV). The file is encrypted using AES-256-GCM. Simultaneously, a SHA-256 hash of the original plaintext file is calculated. The encrypted blob is routed to PostgreSQL, while the SHA-256 hash and file ID are submitted via the gateway to the Fabric smart contract, anchoring the file's exact digital signature permanently on the ledger.

Step 3: Access Request Initiation & Purpose Binding A doctor logs in and requests access to a specific patient's record via the `/api/v1/access/request` endpoint. They must specify a clinical purpose and a required duration (in seconds). The smart contract intercepts this request, validates that the requested record ID actually exists on the ledger, and generates a new access request object in the `REQUESTED` state with a calculated UNIX expiration timestamp.

Step 4: Patient Consent & Real-Time Approval The patient navigates to their dashboard, which polls the backend for pending requests. The patient evaluates the doctor's request and triggers the

`/api/v1/access/{id}/approve` endpoint. The backend submits this transaction to the smart contract, which verifies that the user submitting the approval is the exact cryptographic owner of the record. Once verified, the blockchain state transitions to `APPROVED`, permanently logging the patient's explicit consent.

Step 5: File Retrieval & Dynamic Integrity Verification When an approved doctor attempts to download the medical file, they hit the `/api/v1/records/{id}/download` endpoint. The backend intercepts this and first queries the Fabric ledger using the `CheckAccess()` smart contract function. The ledger confirms if the doctor has an active, non-expired `APPROVED` status. If validated, the backend fetches the AES-encrypted blob from PostgreSQL, decrypts it in memory, and dynamically re-computes its SHA-256 hash. If this newly calculated hash perfectly matches the immutable hash anchored on the ledger during Step 2, the file is securely streamed to the doctor. If a single byte differs, it flags a tampering event and rejects the request (HTTP 500).

Step 6: Real-Time Access Revocation Patient ownership is maintained through the ability to instantly sever access. The patient can trigger the `/api/v1/access/{id}/revoke` endpoint. The smart contract updates the ledger state to `REVOKED`. Because the backend queries the ledger in real-time during Step 5, any subsequent download attempts by the doctor will immediately be blocked and return an HTTP 403 Forbidden error.

Step 7: Immutable Audit Trail Generation Every single lifecycle event (registration, request, approval, download, revocation) generates a permanent block on the Hyperledger Fabric ledger. The `/api/v1/audit/{recordId}` endpoint utilizes Fabric's history query capabilities to retrieve every state change associated with a specific record. This data is parsed and rendered on the frontend

as an immutable, tamper-resistant audit timeline complete with transaction IDs and timestamps.

7. Key Source Code

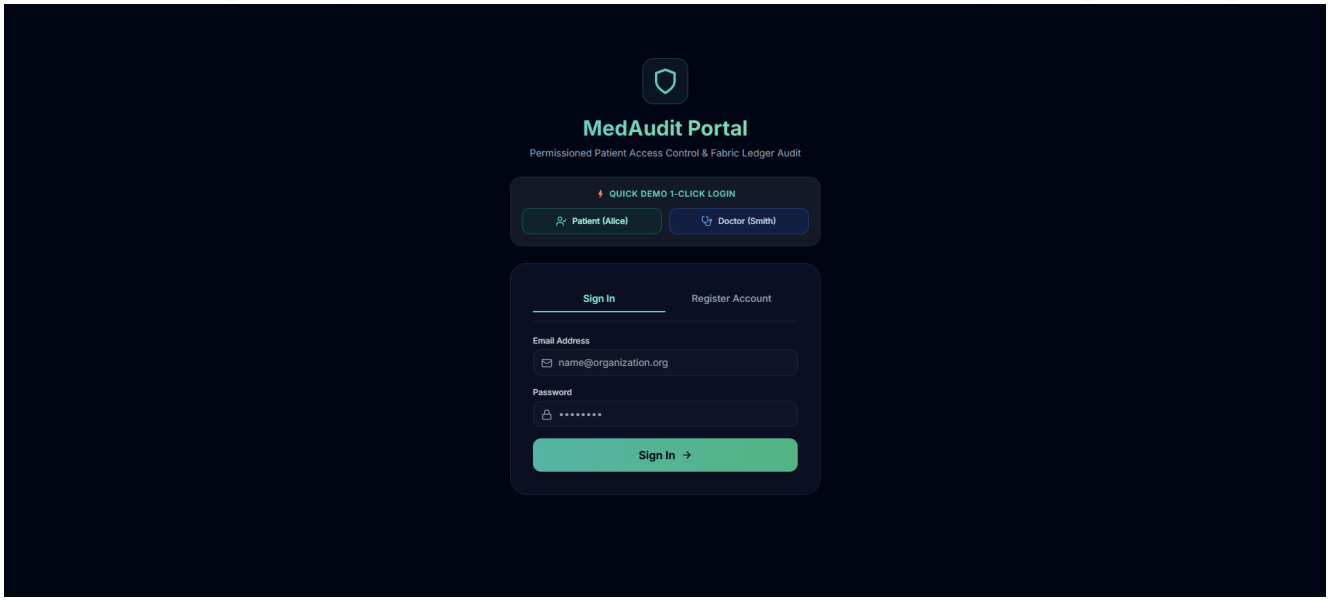
The system's core access logic is governed by a robust TypeScript smart contract (Chaincode) executing within the Hyperledger Fabric peer container. The chaincode interacts with the ledger state using Fabric's native `PutState` and `GetState` APIs.

- `RegisterRecord(recordId, patientId, fileHash)`: This function serves as the genesis of a medical record's lifecycle. It accepts the unique record identifier and the SHA-256 cryptographic hash. It uses `GetState` to ensure the `recordId` does not already exist, preventing duplicate registrations or overwrites. It then uses `PutState` to anchor the hash on the ledger and records an immutable `RECORD_REGISTERED` audit event.
- `RequestAccess(requestId, recordId, patientId, requesterId, purpose, durationSeconds)`: This function handles the initiation of data sharing. It validates that the underlying record exists, computes the expiration boundary by adding `durationSeconds` to the current block timestamp, and writes an access request object to the ledger in the `REQUESTED` state.
- `ApproveAccess(requestId)`: This critical function transitions the state of a specific request to `APPROVED`. To prevent privilege escalation, it strictly validates that the transaction invoker (derived from the client identity passing the transaction) matches the exact `patientId` registered as the owner of the record.
- `DenyAccess(requestId)`: If a patient rejects a doctor's request, this function transitions the state to `DENIED`. It acts as a permanent block, disallowing any future state transitions or file retrievals for that specific request ID, ensuring the doctor must submit a completely new request if access is needed later.
- `RevokeAccess(requestId)`: This function empowers the patient with absolute control. It transitions an active `APPROVED` state directly to `REVOKED`. Because the backend architecture relies on the ledger as the single source of truth, this instantly severs the doctor's access rights across the entire application ecosystem.
- `CheckAccess(recordId, requesterId)`: This is a high-frequency read-only evaluation function. Before the FastAPI backend releases any decrypted data, it calls `CheckAccess()`. The function evaluates the ledger state to verify that an `APPROVED` authorization exists between the `requesterId` and the `recordId`, and calculates if the current block timestamp has exceeded the agreed-upon expiration limit.
- `GetAuditTrail(recordId)`: Utilizing Fabric's `GetHistoryForKey` API, this function queries the historical block state rather than just the current state. It compiles an array of every modification made to the record, returning the complete, unforgeable ledger of all access events for administrative and patient oversight.

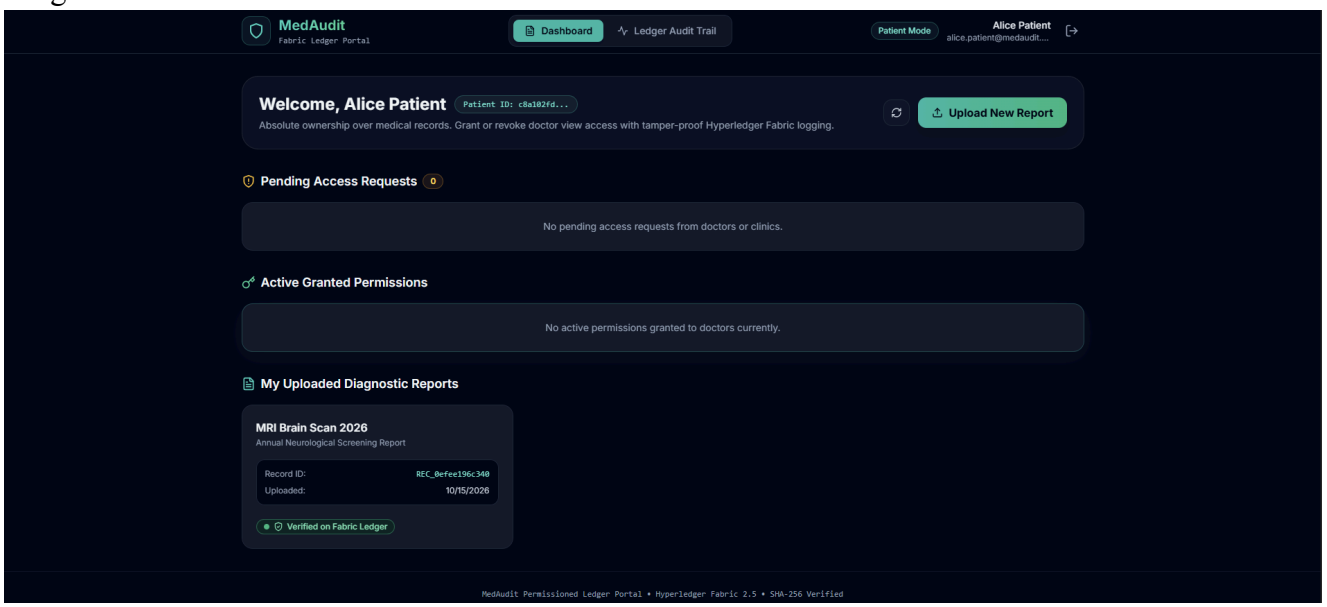
8.Results

The MedAudit application was fully tested through an automated integration suite achieving a 100% success rate across user registration, cryptographic anchoring, access requests, file retrieval, and revocation workflows.

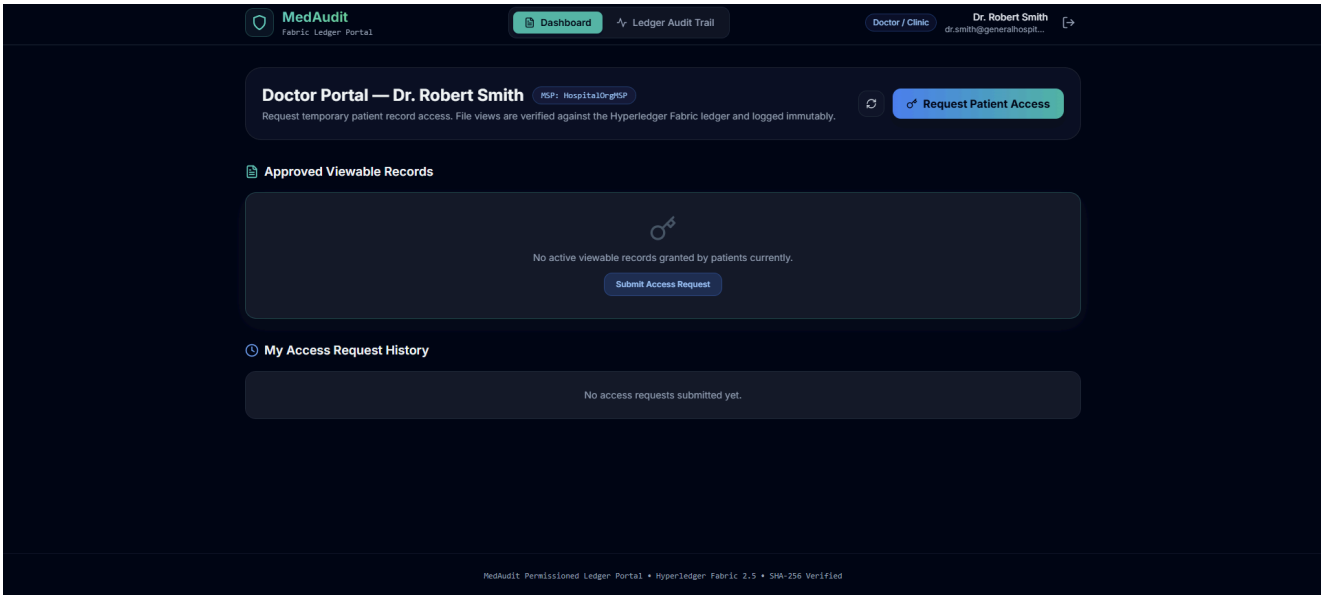
1. Role-Based Authentication & Gateway Entry The MedAudit landing page securely initializes user identities by providing role selector cards for Patients and Doctors alongside a quick login portal.



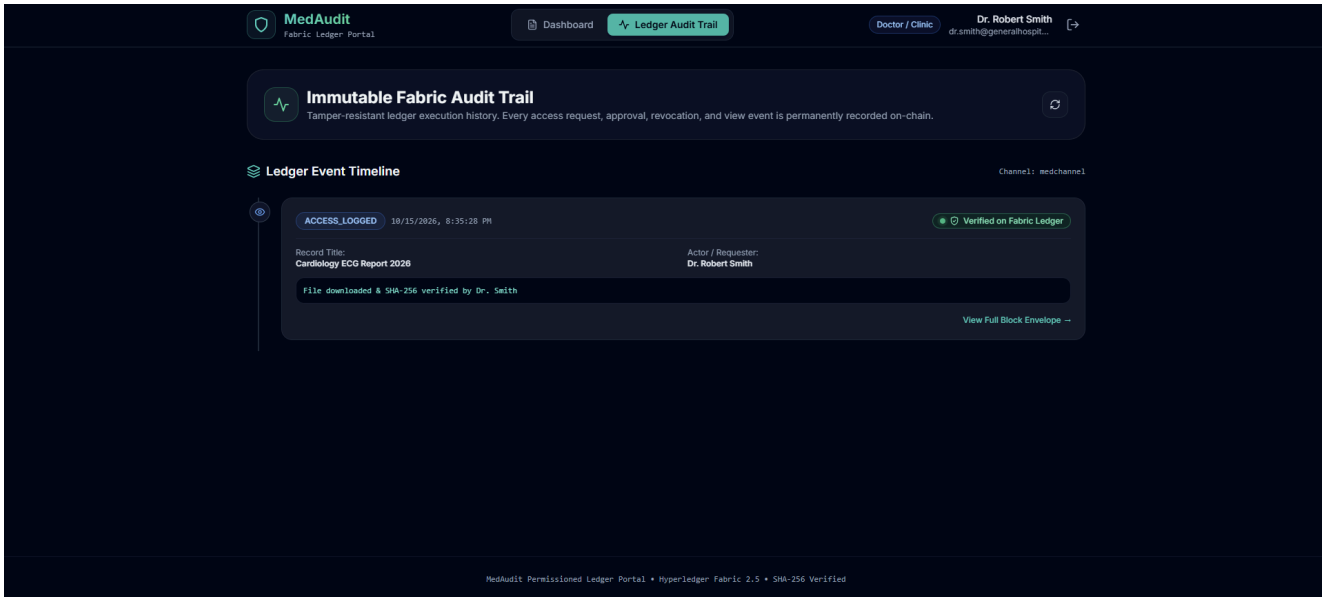
2. Patient Dashboard & Encrypted Record Anchoring The patient interface displays absolute ownership over medical records, showing uploaded diagnostic reports (like an MRI Brain Scan) alongside a verification badge confirming the SHA-256 cryptographic hash is anchored to the Fabric Ledger.



3. Doctor Access Request Portal The Doctor Portal allows medical professionals to view active granted permissions and provides an interface to submit new requests for temporary patient record access, which are subsequently verified against the ledger.



4. Immutable Fabric Audit Trail The system generates a tamper-resistant ledger execution history, permanently recording every access request, approval, revocation, and view event on-chain, explicitly noting when a file is downloaded and its SHA-256 hash verified.



9. Future Scope

The proposed MedAudit system establishes a highly secure, foundational architecture that can be significantly extended to address broader healthcare interoperability, massive scalability, and advanced cryptographic challenges. Future iterations of the platform can integrate the following complex clinical workflows:

9.1 Zero-Knowledge Proofs (zk-SNARKs) While the current MedAudit system utilizes AES-256-GCM for highly secure full-file encryption, future versions can implement Zero-Knowledge Succinct Non-Interactive Arguments of Knowledge (zk-SNARKs). In traditional healthcare, proving a condition requires revealing the entire medical file. zk-SNARKs introduce a cryptographic paradigm where a patient's client application can generate a complex mathematical proof validating a specific health parameter (e.g., confirming a specific blood type for a transfusion, validating the presence of a penicillin allergy, or proving a negative viral test result) without ever decrypting or revealing the underlying raw medical document. This enforces the ultimate standard of data minimization, allowing medical professionals, employers, or insurance providers to cryptographically verify a binary clinical claim while the patient maintains absolute privacy over their comprehensive medical history.

9.2 Decentralized Identifiers (DIDs) and Verifiable Credentials To scale MedAudit into a zero-trust network spanning across disparate, global healthcare consortiums, the system can integrate W3C-standard Decentralized Identifiers (DIDs) paired with Verifiable Credentials (VCs). Currently, identity relies on local hospital databases. In a future iteration, state medical boards or national health authorities could issue cryptographically signed Verifiable Credentials to practitioners. Before a doctor can even submit an access request on the MedAudit network, the Fabric chaincode would automatically parse and verify their DID and credential signature. This mathematically guarantees the requester's active medical license, clinical specialization, and current hospital affiliation in real-time, effectively eliminating the risk of social engineering, unauthorized impersonation, or rogue actors attempting to breach patient privacy networks.

9.3 Emergency Override (Break-Glass Protocol) The current system operates on a strict, explicit consent model, which inherently assumes the patient is conscious, aware, and capable of approving requests via their digital dashboard. To accommodate critical, life-threatening clinical scenarios (such as an unconscious patient arriving at a Level 1 trauma center), the architecture must be extended with a "Break-Glass" emergency protocol. This feature would introduce a multi-signature smart contract function allowing immediate access to critical, life-saving records without explicit patient interaction. To mathematically prevent abuse and ensure absolute accountability, this override would require simultaneous, multi-party cryptographic authorizations from highly privileged personnel (e.g., the attending ER trauma physician and an on-duty hospital administrator). Executing a break-glass transaction would instantly decrypt the data, notify the patient's emergency contacts, and log a permanently red-flagged, high-priority event on the Hyperledger Fabric ledger, mandating a strict, post-hoc administrative and legal review to verify the clinical justification of the override.

10. Conclusion

The MedAudit project successfully demonstrates the practical, highly secure implementation of a permissioned blockchain solution tailored specifically for the rigorous demands of healthcare access control. By strategically combining AES-256-GCM off-chain envelope encryption with Hyperledger Fabric's on-chain immutable ledger anchoring, the system effectively neutralizes the primary vulnerabilities inherent in traditional centralized Electronic Health Record databases.

The resulting dual-boundary architecture shifts the paradigm of medical data management by achieving absolute, patient-centric consent control. It completely eliminates central points of failure, ensuring that malicious database breaches yield only unusable ciphertext. Crucially, the system ensures strict regulatory privacy compliance with frameworks like HIPAA and GDPR by mathematically guaranteeing that raw medical data never touches the distributed ledger. By executing access logic via deterministic smart contracts and securely logging every lifecycle event, MedAudit provides complete non-repudiation of access logs, fostering an environment of total transparency, unparalleled security, and cryptographically guaranteed trust between patients and healthcare providers.